

1. Write a C program that contains a string (char pointer) with a value Hello world. The program should XOR each character in this string with 0 and displays the result.

```
#include<stdio.h>
#include<string.h>
int main()
{
char str[]="Hello World";
char str1[11];
int i,len;
len=strlen(str);
for(i=0;i<len;i++)
{
str1[i]=str[i]^0;
printf("%c",str1[i]);
}
printf("\n");
}
```

Output

Hello World

2. Write a C program that contains a string (char pointer) with a value "Hello world". The program should AND or and XOR each character in this string with 127 and display the result.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[] = "Hello World";
    char str1[12];
    char str2[12];
    char str3[12];
    int i, len;

    len = strlen(str);

    for(i = 0; i < len; i++)
    {
        str1[i] = str[i] & 127;
        printf("%c", str1[i]);
    }
    str1[len] = '\0';

    printf("\n");

    for(i = 0; i < len; i++)
    {
        str2[i] = str[i] | 127;
        printf("%c", str2[i]);
    }
    str2[len] = '\0';

    printf("\n");

    for(i = 0; i < len; i++)
    {
        str3[i] = str[i] ^ 127;
        printf("%c", str3[i]);
    }
    str3[len] = '\0';

    return 0;
}
```

Write a Java program to perform encryption and decryption using the following algorithms:

1. a. Caesar cipher b. Substitution cipher c. Hill Cipher

Caesar Cipher

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.Scanner;

public class CaesarCipher {

    static Scanner sc = new Scanner(System.in);
    static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

    public static void main(String[] args) throws IOException {

        System.out.print("Enter any String: ");
        String str = br.readLine();

        System.out.print("\nEnter the Key: ");
        int key = sc.nextInt();

        String encrypted = encrypt(str, key);
        System.out.println("\nEncrypted String is: " + encrypted);

        String decrypted = decrypt(encrypted, key);
        System.out.println("\nDecrypted String is: " + decrypted);

        System.out.println();
    }

    public static String encrypt(String str, int key) {

        String encrypted = "";

        for (int i = 0; i < str.length(); i++) {

            int c = str.charAt(i);

            if (Character.isUpperCase(c)) {
                c = c + (key % 26);
                if (c > 'Z')
                    c = c - 26;
            } else if (Character.isLowerCase(c)) {
                c = c + (key % 26);
                if (c > 'z')
                    c = c - 26;
            }

            encrypted += (char) c;
        }
    }
}
```

```

    }

    return encrypted;
}

public static String decrypt(String str, int key) {

    String decrypted = "";

    for (int i = 0; i < str.length(); i++) {

        int c = str.charAt(i);

        if (Character.isUpperCase(c)) {
            c = c - (key % 26);
            if (c < 'A')
                c = c + 26;
        } else if (Character.isLowerCase(c)) {
            c = c - (key % 26);
            if (c < 'a')
                c = c + 26;
        }

        decrypted += (char) c;
    }

    return decrypted;
}
}

```

Substitution cipher

```
import java.io.*;

import java.util.*;

public class SubstitutionCipher {

    static Scanner sc = new Scanner(System.in);

    static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

    public static void main(String[] args) throws IOException {

        String a = "abcdefghijklmnopqrstuvwxyzH";

        String b = "zyxwvutsrqponmlkjihgfedcbaS";

        System.out.print("Enter any string: ");

        String str = br.readLine();

        String decrypt = "";

        char c;

        for(int i=0;i<str.length();i++)

        {

            c = str.charAt(i);

            int j = a.indexOf(c);

            decrypt = decrypt+b.charAt(j);

        }

        System.out.println("The encrypted data is: " +decrypt);

    }

}
```

Hill Cipher

```
import java.util.Scanner;

public class HillCipher {

    static final int SIZE = 3;

    // Function to find modular inverse of determinant
    static int modInverse(int a, int m) {

        a = ((a % m) + m) % m;

        for (int x = 1; x < m; x++) {

            if ((a * x) % m == 1)

                return x;

        }

        return -1;

    }

    // Determinant of 3x3 matrix
    static int determinant(int[][] key) {

        int det = key[0][0] * (key[1][1] * key[2][2] - key[1][2] * key[2][1])

            - key[0][1] * (key[1][0] * key[2][2] - key[1][2] * key[2][0])

            + key[0][2] * (key[1][0] * key[2][1] - key[1][1] * key[2][0]);

        det %= 26;

        if (det < 0)

            det += 26;

        return det;

    }

    // Find inverse matrix modulo 26
    static int[][] inverse(int[][] key) {
```

```

int det = determinant(key);

int detInv = modInverse(det, 26);

if (detInv == -1) {

    System.out.println("Key matrix is not invertible modulo 26.");

    System.exit(0);

}

int[][] adj = new int[3][3];

adj[0][0] = key[1][1] * key[2][2] - key[1][2] * key[2][1];
adj[0][1] = -(key[1][0] * key[2][2] - key[1][2] * key[2][0]);
adj[0][2] = key[1][0] * key[2][1] - key[1][1] * key[2][0];
adj[1][0] = -(key[0][1] * key[2][2] - key[0][2] * key[2][1]);
adj[1][1] = key[0][0] * key[2][2] - key[0][2] * key[2][0];
adj[1][2] = -(key[0][0] * key[2][1] - key[0][1] * key[2][0]);
adj[2][0] = key[0][1] * key[1][2] - key[0][2] * key[1][1];
adj[2][1] = -(key[0][0] * key[1][2] - key[0][2] * key[1][0]);
adj[2][2] = key[0][0] * key[1][1] - key[0][1] * key[1][0];

// Transpose and multiply by determinant inverse

int[][] inv = new int[3][3];

for (int i = 0; i < SIZE; i++) {

    for (int j = 0; j < SIZE; j++) {

        inv[i][j] = adj[j][i];

        inv[i][j] = (inv[i][j] * detInv) % 26;

        if (inv[i][j] < 0)

            inv[i][j] += 26;

    }

}

```

```

        return inv;
    }

    // Matrix multiplication modulo 26
    static int[] multiply(int[][] key, int[] msg) {
        int[] result = new int[SIZE];
        for (int i = 0; i < SIZE; i++) {
            result[i] = 0;
            for (int j = 0; j < SIZE; j++) {
                result[i] += key[i][j] * msg[j];
            }
            result[i] %= 26;
        }
        return result;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] key = new int[3][3];
        System.out.println("Enter 3x3 key matrix:");
        for (int i = 0; i < SIZE; i++)
            for (int j = 0; j < SIZE; j++)
                key[i][j] = sc.nextInt();
        System.out.print("Enter 3-letter plaintext: ");
        String text = sc.next().toLowerCase();
        if (text.length() != 3) {
            System.out.println("Plaintext must contain exactly 3 letters.");
        }
    }
}

```



```

        return;
    }

    int[] plain = new int[SIZE];

    for (int i = 0; i < SIZE; i++)

        plain[i] = text.charAt(i) - 'a';

    // Encryption

    int[] cipher = multiply(key, plain);

    System.out.print("\nEncrypted Text: ");

    for (int x : cipher)

        System.out.print((char) (x + 'a'));

    // Decryption

    int[][] invKey = inverse(key);

    int[] decrypted = multiply(invKey, cipher);

    System.out.print("\nDecrypted Text: ");

    for (int x : decrypted)

        System.out.print((char) (x + 'a'));

    System.out.println();

    sc.close();

}

}

```

4. Write a C/JAVA program to implement the DES algorithm logic.

```
import java.io.BufferedReader;

import java.io.InputStreamReader;

import java.security.spec.KeySpec;

import java.util.Base64;

import javax.crypto.Cipher;

import javax.crypto.SecretKey;

import javax.crypto.SecretKeyFactory;

import javax.crypto.spec.DESedeKeySpec;

public class DES {

    private static final String UNICODE_FORMAT = "UTF8";

    public static final String DESEDE_ENCRYPTION_SCHEME = "DESEde";

    private KeySpec myKeySpec;

    private SecretKeyFactory mySecretKeyFactory;

    private Cipher cipher;

    private byte[] keyAsBytes;

    private String myEncryptionKey;

    private String myEncryptionScheme;

    private SecretKey key;

    static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

    public DES() throws Exception {

        myEncryptionKey = "ThisIsSecretEncryptionKey";

        myEncryptionScheme = DESEDE_ENCRYPTION_SCHEME;

        keyAsBytes = myEncryptionKey.getBytes(UNICODE_FORMAT);

        myKeySpec = new DESedeKeySpec(keyAsBytes);

        mySecretKeyFactory = SecretKeyFactory.getInstance(myEncryptionScheme);
```

```

    cipher = Cipher.getInstance(myEncryptionScheme);

    key = mySecretKeyFactory.generateSecret(myKeySpec);
}

public String encrypt(String unencryptedString) {
    try {
        cipher.init(Cipher.ENCRYPT_MODE, key);

        byte[] plainText = unencryptedString.getBytes(UNICODE_FORMAT);

        byte[] encryptedText = cipher.doFinal(plainText);

        return Base64.getEncoder().encodeToString(encryptedText);
    } catch (Exception e) {
        e.printStackTrace();
    }

    return null;
}

public String decrypt(String encryptedString) {
    try {
        cipher.init(Cipher.DECRYPT_MODE, key);

        byte[] encryptedText = Base64.getDecoder().decode(encryptedString);

        byte[] plainText = cipher.doFinal(encryptedText);

        return new String(plainText, UNICODE_FORMAT);
    } catch (Exception e) {
        e.printStackTrace();
    }

    return null;
}

```

```
public static void main(String[] args) throws Exception {  
    System.out.print("Enter the string: ");  
    DES myEncryptor = new DES();  
    String stringToEncrypt = br.readLine();  
    String encrypted = myEncryptor.encrypt(stringToEncrypt);  
    String decrypted = myEncryptor.decrypt(encrypted);  
    System.out.println("\nString To Encrypt : " + stringToEncrypt);  
    System.out.println("Encrypted Value  : " + encrypted);  
    System.out.println("Decrypted Value  : " + decrypted);  
}  
}
```

5. Write a C/JAVA program to implement the Blowfish algorithm logic.

```
import java.util.*;

import java.security.Key;

import javax.crypto.Cipher;

import javax.crypto.KeyGenerator;

import java.util.Base64;


public class BlowFish {


    public static void main(String[] args) throws Exception {


        // Generate Blowfish key

        KeyGenerator keyGenerator =

            KeyGenerator.getInstance("Blowfish");


        keyGenerator.init(128);


        Key secretKey = keyGenerator.generateKey();


        // Create Cipher

        Cipher cipher =

            Cipher.getInstance("Blowfish/CFB/NoPadding");


        cipher.init(Cipher.ENCRYPT_MODE, secretKey);


        // Get Initialization Vector
```

```
byte[] iv = cipher.getIV();

System.out.println(
    "Initialization Vector of the Cipher: " +
    Base64.getEncoder().encodeToString(iv)
);

// Input from user
Scanner sc = new Scanner(System.in);

System.out.print("Enter text to encrypt: ");
String text = sc.nextLine();

// Encrypt
byte[] encrypted =
    cipher.doFinal(text.getBytes());

// Display encrypted text
System.out.println(
    "Encrypted Text: " +
    Base64.getEncoder().encodeToString(encrypted)
);

sc.close();
}
}
```

